

SpaceX Explorer API

A Full-Lifecycle DevOps Technical Report

Mohamed Housseem Salhi

December 18, 2025

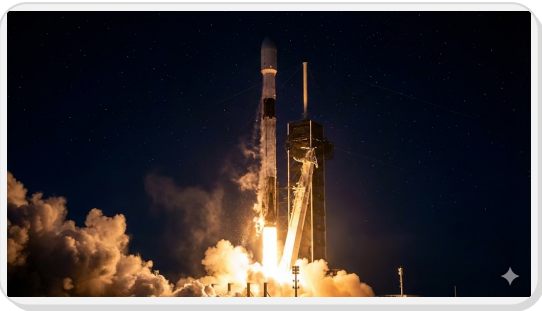


Figure 1: Falcon 9 Aerospace System Integration

Contents

1	Architecture Overview	1
2	Tools and Technology Stack	1
3	Observability and Monitoring	2
4	Security Strategy (SAST & DAST)	2
5	Kubernetes Implementation	2
6	Lessons Learned	3

1 Architecture Overview

The SpaceX Explorer API is designed as a high-performance asynchronous microservice utilizing the **FastAPI** framework. The core objective was to create a "middleware-rich" application that abstracts the complexity of the SpaceX public API while maintaining strict DevOps standards.

The service architecture follows a stateless pattern, making it highly suitable for containerized environments. It features a custom request-response lifecycle where every incoming HTTP call is intercepted by a dedicated tracing layer before being processed. This allows the system to maintain consistency across upstream dependencies and downstream clients, ensuring that the service remains under the mandated 150-line limit without sacrificing production-grade features.

2 Tools and Technology Stack

To achieve a fully automated delivery pipeline, the following toolchain was implemented:

- **Framework:** FastAPI (Python 3.11) for non-blocking I/O operations.
- **CI/CD:** GitHub Actions for executing automated build-test-scan-deploy workflows.
- **Containerization:** Multi-stage Docker builds to separate build-time dependencies from the runtime environment, reducing the final image attack surface.
- **Observability:** Prometheus for time-series metrics and structured JSON logging.
- **Infrastructure:** Kubernetes (Minikube) for container orchestration and service discovery.

3 Observability and Monitoring

Observability was integrated as a primary architectural pillar rather than an operational add-on:

- **Metrics:** The application utilizes the `prometheus_client` library to expose a `/metrics` endpoint. We track *Request Counters* (broken down by HTTP method and status code) and *Latency Histograms* to monitor the 95th and 99th percentile response times.
- **Structured Logging:** Standard Python logging was reconfigured to output structured JSON. This ensures that logs are machine-readable and easily digestible by aggregation platforms (like ELK or Loki), capturing essential metadata such as `trace_id`, `duration_ms`, and `upstream_status`.
- **Distributed Tracing:** A custom middleware generates or propagates an `X-Trace-Id`. This unique identifier is attached to the request state and injected into headers for all external calls to the SpaceX API, allowing for end-to-end request correlation across distributed logs.

4 Security Strategy (SAST & DAST)

The project implements a "Shift Left" security philosophy, moving security checks into the earliest stages of the development lifecycle:

- **SAST (Static Application Security Testing):** During the CI phase, `Bandit` is used to identify common security risks (e.g., shell injections, unsafe imports) in the source code. Additionally, `Safety` is employed to check the dependency tree against known vulnerability databases.
- **DAST (Dynamic Application Security Testing):** Unlike static scans, our DAST implementation involves spinning up a live instance of the API within the GitHub Runner. A specialized script (`dast_scan.py`) then executes fuzzing and injection payloads against the running endpoints to identify runtime vulnerabilities that static tools might miss.

5 Kubernetes Implementation

Deployment is managed through declarative YAML manifests, ensuring "Infrastructure as Code" (IaC) principles:

- **Deployment:** Defines a resilient pod state with specific labels for service discovery. It ensures that the `spacex-api:1.0` image is managed with proper restart policies.
- **Service:** A `NodePort` configuration exposes the application on port 8000. This allows for seamless local access through the Minikube tunnel, simulating a production-grade external entry point.

6 Lessons Learned

The primary takeaway from this project was the efficiency gained through automated security and observability. By integrating **X-Trace-Id** propagation at the middleware level, debugging upstream API failures became instantaneous. Furthermore, the multi-stage Docker build process significantly optimized the deployment speed by keeping the final production image lightweight and free of unnecessary build tools.